```
List(12,14).map(_ * 3)

List(36,42)
```

Here we're showing the result of each substitution performed to evaluate our expression. For example, to go from the first line to the second, we've replaced `List(1,2,3,4).map(_ + 10)` with `List(11,12,13,14)`, based on the definition of `map`.[2] This view makes it clear how the calls to `map` and `filter` each perform their own traversal of the input and allocate lists for the output. Wouldn't it be nice if we could somehow fuse sequences of transformations like this into a single pass and avoid creating temporary data structures? We could rewrite the code into a `while` loop by hand, but ideally we'd like to have this done automatically while retaining the same high-level compositional style. We want to compose our programs using higher-order functions like `map` and `filter` instead of writing monolithic loops.

It turns out that we can accomplish this kind of automatic loop fusion through the use of *non-strictness* (or, less formally, *laziness*). In this chapter, we'll explain what exactly this means, and we'll work through the implementation of a lazy list type that fuses sequences of transformations. Although building a "better" list is the motivation for this chapter, we'll see that non-strictness is a fundamental technique for improving on the efficiency and modularity of functional programs in general.

5.1 Strict and non-strict functions

Before we get to our example of lazy lists, we need to cover some basics. What are strictness and non-strictness, and how are these concepts expressed in Scala?

Non-strictness is a property of a function. To say a function is non-strict just means that the function may choose *not* to evaluate one or more of its arguments. In contrast, a *strict* function always evaluates its arguments. Strict functions are the norm in most programming languages, and indeed most languages only support functions that expect their arguments fully evaluated. Unless we tell it otherwise, any function definition in Scala will be strict (and all the functions we've defined so far have been strict). As an example, consider the following function:

```
def square(x: Double): Double = x * x
```

When you invoke `square(41.0 + 1.0)`, the function `square` will receive the evaluated value of `42.0` because it's strict. If you invoke `square(sys.error("failure"))`, you'll get an exception before `square` has a chance to do anything, since the `sys.error("failure")` expression will be evaluated before entering the body of `square`.

Although we haven't yet learned the syntax for indicating non-strictness in Scala, you're almost certainly familiar with the concept. For example, the short-circuiting Boolean functions `&&` and `||`, found in many programming languages including Scala, are non-strict. You may be used to thinking of `&&` and `||` as built-in syntax, part of the

[2] With program traces like these, it's often more illustrative to not fully trace the evaluation of every subexpression. In this case, we've omitted the full expansion of `List(1,2,3,4).map(_ + 10)`. We could "enter" the definition of `map` and trace its execution, but we chose to omit this level of detail here.

language, but you can also think of them as functions that may choose not to evaluate their arguments. The function `&&` takes two `Boolean` arguments, but only evaluates the second argument if the first is `true`:

```
scala> false && { println("!!"); true } // does not print anything
res0: Boolean = false
```

And `||` only evaluates its second argument if the first is `false`:

```
scala> true || { println("!!"); false } // doesn't print anything either
res1: Boolean = true
```

Another example of non-strictness is the `if` control construct in Scala:

```
val result = if (input.isEmpty) sys.error("empty input") else input
```

Even though `if` is a built-in language construct in Scala, it can be thought of as a function accepting three parameters: a condition of type `Boolean`, an expression of some type `A` to return in the case that the condition is `true`, and another expression of the same type `A` to return if the condition is `false`. This `if` function would be non-strict, since it won't evaluate all of its arguments. To be more precise, we'd say that the `if` function is strict in its condition parameter, since it'll always evaluate the condition to determine which branch to take, and non-strict in the two branches for the `true` and `false` cases, since it'll only evaluate one or the other based on the condition.

In Scala, we can write non-strict functions by accepting some of our arguments unevaluated. We'll show how this is done explicitly just to illustrate what's happening, and then show some nicer syntax for it that's built into Scala. Here's a non-strict `if` function:

```
def if2[A](cond: Boolean, onTrue: () => A, onFalse: () => A): A =
  if (cond) onTrue() else onFalse()

if2(a < 22,
  () => println("a"),
  () => println("b")
)
```

The function literal syntax for creating a `() => A`

The arguments we'd like to pass unevaluated have a `() =>` immediately before their type. A value of type `() => A` is a function that accepts zero arguments and returns an `A`.[3] In general, the unevaluated form of an expression is called a *thunk*, and we can *force* the thunk to evaluate the expression and get a result. We do so by invoking the function, passing an empty argument list, as in `onTrue()` or `onFalse()`. Likewise, callers of `if2` have to explicitly create thunks, and the syntax follows the same conventions as the function literal syntax we've already seen.

Overall, this syntax makes it very clear what's happening—we're passing a function of no arguments in place of each non-strict parameter, and then explicitly calling this function to obtain a result in the body. But this is such a common case that Scala provides some nicer syntax:

[3] In fact, the type `() => A` is a syntactic alias for the type `Function0[A]`.

```
def if2[A](cond: Boolean, onTrue: => A, onFalse: => A): A =
  if (cond) onTrue else onFalse
```

The arguments we'd like to pass unevaluated have an arrow `=>` immediately before their type. In the body of the function, we don't need to do anything special to evaluate an argument annotated with `=>`. We just reference the identifier as usual. Nor do we have to do anything special to call this function. We just use the normal function call syntax, and Scala takes care of wrapping the expression in a thunk for us:

```
scala> if2(false, sys.error("fail"), 3)
res2: Int = 3
```

With either syntax, an argument that's passed unevaluated to a function will be evaluated once for each place it's referenced in the body of the function. That is, Scala won't (by default) cache the result of evaluating an argument:

```
scala> def maybeTwice(b: Boolean, i: => Int) = if (b) i+i else 0
maybeTwice: (b: Boolean, i: => Int)Int

scala> val x = maybeTwice(true, { println("hi"); 1+41 })
hi
hi
x: Int = 84
```

Here, `i` is referenced twice in the body of `maybeTwice`, and we've made it particularly obvious that it's evaluated each time by passing the block `{println("hi"); 1+41}`, which prints `hi` as a side effect before returning a result of `42`. The expression `1+41` will be computed twice as well. We can cache the value explicitly if we wish to only evaluate the result once, by using the `lazy` keyword:

```
scala> def maybeTwice2(b: Boolean, i: => Int) = {
     |   lazy val j = i
     |   if (b) j+j else 0
     | }
maybeTwice: (b: Boolean, i: => Int)Int

scala> val x = maybeTwice2(true, { println("hi"); 1+41 })
hi
x: Int = 84
```

Adding the `lazy` keyword to a `val` declaration will cause Scala to delay evaluation of the right-hand side of that `lazy val` declaration until it's first referenced. It will also cache the result so that subsequent references to it don't trigger repeated evaluation.

Formal definition of strictness

If the evaluation of an expression runs forever or throws an error instead of returning a definite value, we say that the expression doesn't *terminate*, or that it evaluates to *bottom*. A function `f` is *strict* if the expression `f(x)` evaluates to bottom for all `x` that evaluate to bottom.